UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR

POSGRADOS

Maestría en

SOFTWARE

Asignatura:

Patrones de Diseño de Software

Actividad #: 03

Patrones de comportamiento

Docente:
Mauricio Ortiz Ochoa

Elaborado por (Grupo 02):
Espinoza Adrian
Jara Pablo
Lara Miguel
Torres Carina
Uyaguari Sebastián

Fecha de entrega:
jueves, 29 de octubre de 2026

Repositorio de código:
https://github.com/carytnas/PDS_Actividad03_Grupo02

Actividad de aprendizaje #: 03

CC BY NC ND

Introducción

Este informe presenta la resolución de la Actividad 03 del Grupo 02. De los patrones de comportamiento propuestos elegimos Mediator y Command porque atacan dos problemas distintos que aparecen con frecuencia en sistemas reales: el primero ordena la forma en que se comunican varios objetos que compiten por un mismo recurso; el segundo convierte una petición en un objeto que se puede asignar, registrar, deshacer o agrupar.

Para Mediator modelamos una torre de control que administra una sola pista para vuelos comerciales y sanitarios. Para Command, un control remoto programable que acciona la luz y el ventilador de una sala de reuniones, con historial para deshacer y escenas predefinidas. Ambas soluciones están implementadas en Java 8, cuentan con pruebas unitarias en JUnit 5 y se encuentran en el repositorio del grupo.

Los dos patrones se desarrollan con la misma secuencia: escenario, problema de diseño, justificación frente a otras alternativas, participantes, modelo UML, código, validación y uso crítico de la IA. Al final se presentan las pruebas automatizadas, una síntesis comparativa y las conclusiones.

Patrón 01: Mediator – Torre de control de tráfico aéreo

Escenario / Motivación

Un aeródromo regional opera con una sola pista. En horas pico varias aeronaves se aproximan al mismo tiempo y todas necesitan aterrizar, pero la pista admite una sola a la vez. Los vuelos comerciales se atienden por orden de llegada; los vuelos sanitarios, que trasladan pacientes, tienen prioridad sobre los regulares que estén esperando, aunque nunca interrumpen un aterrizaje en curso.

Cada aeronave sabe pedir autorización, esperar y aterrizar. Lo que no debería saber es quién más está en el aire, en qué orden van los demás ni qué regla de prioridad se aplica. Esa información y esas decisiones le corresponden al control de tránsito, no a cada avión.

Problema de diseño identificado

Sin un coordinador, cada aeronave tiene que conocer y consultar a las demás antes de usar la pista (Figura 1). Con n aeronaves aparecen hasta n·(n−1) dependencias y la regla de turnos queda repartida entre todas las clases, porque cada una debe decidir por su cuenta si le toca. Introducir la prioridad sanitaria en ese

esquema implicaría modificar todos los tipos de aeronave, y agregar un tipo nuevo obligaría a revisar los existentes.

El problema, por tanto, no está en lo que hace cada aeronave sino en cómo interactúan. Lo que hay que organizar es quién usa la pista, en qué orden y quién avisa a quién cuando la situación cambia.

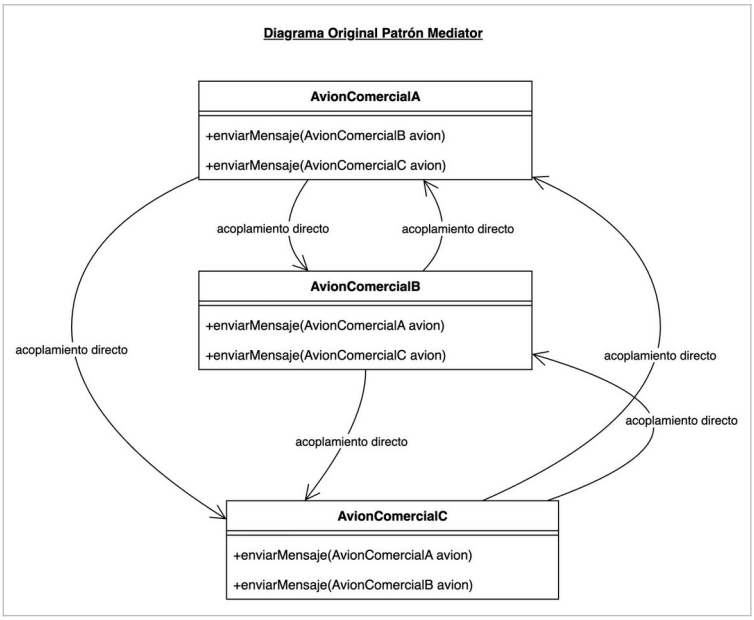


Figura 1: *Diagrama original, sin Mediator*

Justificación del patrón

Mediator encapsula en un objeto la forma en que interactúa un conjunto de objetos, de modo que estos no se refieran entre sí de manera explícita y su interacción pueda variar de forma independiente (Gamma et al., 1994). Es lo que pide el escenario: la torre concentra el estado de la pista, las colas y la regla de prioridad, mientras que las aeronaves solo conocen la interfaz ITorreControl.

Alternativas consideradas y descartadas:

- **Comunicación directa.** Es la Figura 1: el acoplamiento crece con cada aeronave que se agrega y la regla de turnos se duplica en todas las clases.
- **Observer.** Sirve para difundir el evento «pista libre», pero no decide a quién le corresponde. Todas las aeronaves en espera recibirían el aviso a la vez y tendrían que resolver el turno entre ellas, que es justamente el problema de partida.
- **Chain of Responsibility.** Hace pasar una solicitud por una cadena hasta que alguien la atiende. Aquí siempre la atiende el mismo objeto, y ese objeto necesita ver el estado completo (pista y colas) para decidir.
- **Facade.** Simplifica el acceso a un subsistema, pero la comunicación va en un solo sentido. La torre, en cambio, también se dirige a las aeronaves (recibirAutorizacion, recibirEspera); esa comunicación de ida y vuelta es la que caracteriza a Mediator.

Participantes del patrón

- **Mediator → ITorreControl:** declara las operaciones con las que un colega se comunica con la torre: registrarAeronave(), solicitarAterrizaje() y liberarPista().
- **ConcreteMediator → TorreControlConcreta:** mantiene el registro de aeronaves, la cola prioritaria, la cola regular y la aeronave en pista. Decide si autoriza o encola, aplica la prioridad sanitaria y, cuando la pista se libera, autoriza al siguiente. También avisa a los vuelos regulares cuando cambia su posición en la cola.
- **Colleague → Aeronave:** clase abstracta que solo conoce a su mediador. Pide autorización, recibe las respuestas de la torre y aterriza únicamente si fue autorizada. Declara esPrioritaria() para que la torre clasifique la solicitud sin preguntar por el tipo concreto.
- **ConcreteColleague → AvionComercial y AvionSanitario:** cada uno indica si su vuelo es prioritario. Ninguno conoce al otro.
- **Client → MainMediatorTorreControl:** crea la torre y las aeronaves, las registra y ejecuta la simulación.

La colaboración entre los objetos sigue esta secuencia:

1. El cliente registra cada aeronave en la torre. Si una aeronave no registrada pide pista, la torre la rechaza y no la ingresa a ninguna cola.
2. La aeronave invoca solicitarAutorizacion(), que delega en torre.solicitarAterrizaje(this).
3. Si la pista está libre, la torre la asigna y notifica recibirAutorizacion(). Si está ocupada, encola la solicitud según esPrioritaria() y notifica recibirEspera(posicion). Cuando ingresa un vuelo prioritario, la torre recalcula y comunica la nueva posición a los regulares que esperan.
4. Al aterrizar, la aeronave invoca liberarPista(this). La torre toma al siguiente (primero de la cola prioritaria y, si está vacía, de la regular) y lo autoriza.

En la ejecución (Figura 3), VUELO-LA402 encuentra la pista libre; VUELO-IB612 y VUELO-AA990 quedan en las posiciones 1 y 2. Cuando SANITARIO-HC01 pide pista pasa al primer lugar, y la torre informa a los dos comerciales que retroceden a las posiciones 2 y 3. VUELO-AA990 intenta aterrizar sin autorización y la propia aeronave lo impide. En todo el recorrido ningún avión invoca a otro.

Diagrama de clases UML

Seguimos la notación de Fowler (2004): las propiedades cuyo tipo es otra clase del modelo se dibujan como asociaciones navegables y no se repiten como atributos. Por eso TorreControlConcreta muestra cuatro asociaciones hacia Aeronave, una por cada propiedad del código: listaAeronaves (0..*), colaPrioritaria (0..*), colaEspera (0..*) y aeronaveEnPista (0..1). Las aeronaves navegan hacia ITorreControl (0..* a 1) y no existe ninguna asociación entre aeronaves. El cliente se relaciona con las clases que instancia mediante dependencias «create», y una nota documenta la regla de atención de la torre.

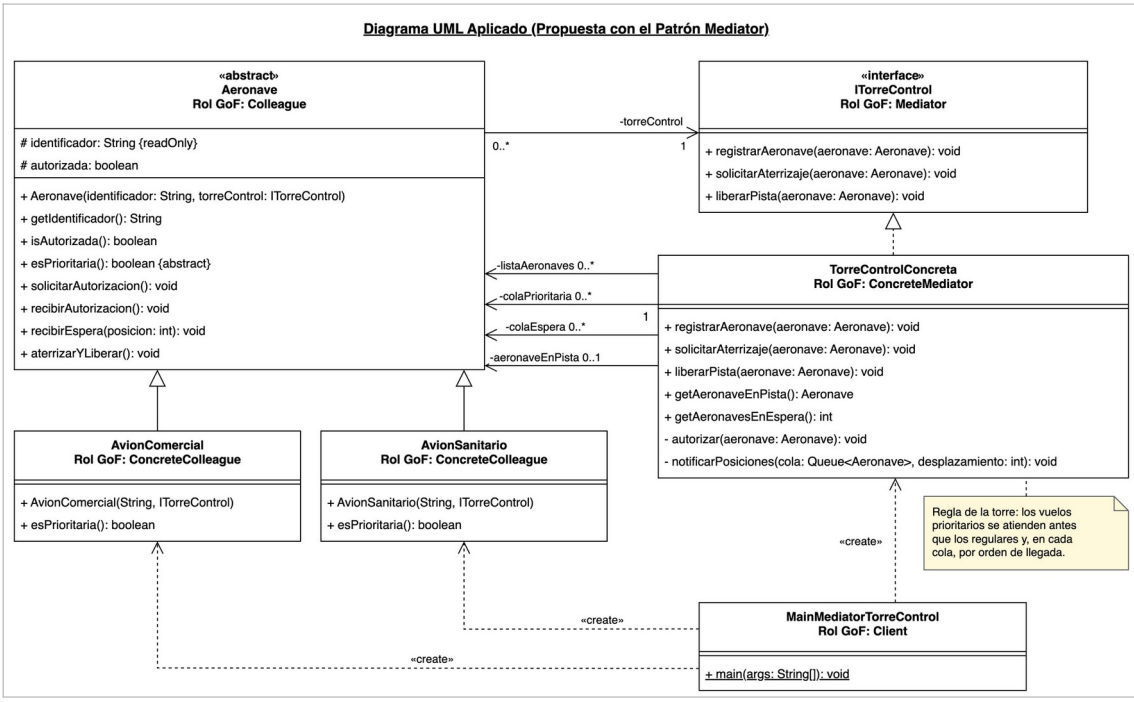


Figura 2: *Diagrama UML con Mediator*

Código fuente

Repositorio / enlace: https://github.com/carytnas/PDS_Actividad03_Grupo02/tree/main/codigo/ejercicios/src/mediator

El núcleo del patrón está en TorreControlConcreta. Se muestran los dos métodos que concentran las reglas: cómo se atiende una solicitud y qué ocurre cuando se libera la pista.

src/mediator/TorreControlConcreta.java (fragmento)

```
public class TorreControlConcreta implements ITorreControl {
  // ... atributos y demás métodos


  @Override
  public void solicitarAterrizaje(Aeronave aeronave) {
    System.out.println("\n[TORRE] Solicitud recibida de la aeronave: " +
aeronave.getIdentificador());


    if (!listaAeronaves.contains(aeronave)) {
```

```
      System.out.println("[TORRE AVISO] La aeronave " + aeronave.getIdentificador() + " no está
registrada.");
      return;
    }
    if (aeronave == aeronaveEnPista || colaPrioritaria.contains(aeronave) ||
colaEspera.contains(aeronave)) {
      System.out.println("[TORRE AVISO] La aeronave " + aeronave.getIdentificador()
          + " ya tiene la pista asignada o está en la cola de espera.");
      return;
    }


    if (aeronaveEnPista == null) {
      autorizar(aeronave);
    } else if (aeronave.esPrioritaria()) {
      colaPrioritaria.add(aeronave);
      System.out.println("[TORRE] Vuelo prioritario " + aeronave.getIdentificador()
          + ": pasa delante de los vuelos regulares en espera.");
      aeronave.recibirEspera(colaPrioritaria.size());
      // Los vuelos regulares retroceden un puesto; la torre se lo comunica a cada uno.
      notificarPosiciones(colaEspera, colaPrioritaria.size());
    } else {
      colaEspera.add(aeronave);
      System.out.println("[TORRE] Pista ocupada actualmente por [" +
aeronaveEnPista.getIdentificador() + "]. "
          + aeronave.getIdentificador() + ", mantenga el patrón de espera.");
      aeronave.recibirEspera(colaPrioritaria.size() + colaEspera.size());
    }
  }


  @Override
  public void liberarPista(Aeronave aeronave) {
    if (aeronaveEnPista != aeronave) {
      System.out.println("[TORRE AVISO] La aeronave " + aeronave.getIdentificador()
          + " intentó liberar la pista pero no estaba asignada a ella.");
      return;
    }
    System.out.println("[TORRE] La aeronave " + aeronave.getIdentificador() + " ha liberado la
pista.");
    aeronaveEnPista = null;


    Aeronave siguiente = colaPrioritaria.poll();
    if (siguiente == null) {
      siguiente = colaEspera.poll();
    }
    if (siguiente != null) {
      autorizar(siguiente);
    }
  }
}
```

```
=== SIMULACIÓN DE TRÁFICO AÉREO (PATRÓN MEDIATOR) ===

[TORRE] Solicitud recibida de la aeronave: VUELO-LA402
[TORRE] Autorizado el aterrizaje para: VUELO-LA402. Pista ocupada.
[VUELO-LA402] Autorización recibida. Procediendo al aterrizaje.

[TORRE] Solicitud recibida de la aeronave: VUELO-IB612
[TORRE] Pista ocupada actualmente por [VUELO-LA402]. VUELO-IB612, mantenga el patrón de espera.
[VUELO-IB612] En patrón de espera (posición 1 en la cola).

[TORRE] Solicitud recibida de la aeronave: VUELO-AA990
[TORRE] Pista ocupada actualmente por [VUELO-LA402]. VUELO-AA990, mantenga el patrón de espera.
[VUELO-AA990] En patrón de espera (posición 2 en la cola).

[TORRE] Solicitud recibida de la aeronave: SANITARIO-HC01
[TORRE] Vuelo prioritario SANITARIO-HC01: pasa delante de los vuelos regulares en espera.
[SANITARIO-HC01] En patrón de espera (posición 1 en la cola).
[VUELO-IB612] En patrón de espera (posición 2 en la cola).
[VUELO-AA990] En patrón de espera (posición 3 en la cola).

[VUELO-AA990] Sin autorización de la torre: no puede aterrizar.

[VUELO-LA402] Realizando maniobra de aterrizaje...
[TORRE] La aeronave VUELO-LA402 ha liberado la pista.
[TORRE] Autorizado el aterrizaje para: SANITARIO-HC01. Pista ocupada.
[SANITARIO-HC01] Autorización recibida. Procediendo al aterrizaje.

[SANITARIO-HC01] Realizando maniobra de aterrizaje...
[TORRE] La aeronave SANITARIO-HC01 ha liberado la pista.
[TORRE] Autorizado el aterrizaje para: VUELO-IB612. Pista ocupada.
[VUELO-IB612] Autorización recibida. Procediendo al aterrizaje.

[VUELO-IB612] Realizando maniobra de aterrizaje...
[TORRE] La aeronave VUELO-IB612 ha liberado la pista.
[TORRE] Autorizado el aterrizaje para: VUELO-AA990. Pista ocupada.
[VUELO-AA990] Autorización recibida. Procediendo al aterrizaje.

[VUELO-AA990] Realizando maniobra de aterrizaje...
[TORRE] La aeronave VUELO-AA990 ha liberado la pista.
```

Figura 3: *Salida de MainMediatorTorreControl*

Validación de la solución

- **Qué resolvió:** las reglas de uso de la pista están en un solo lugar y las aeronaves dejaron de depender unas de otras. Cada aeronave mantiene una única dependencia, hacia ITorreControl, en lugar de n−1.
- **Qué mejoró respecto de la solución directa:** lo comprobamos al extender el escenario. Agregar AvionSanitario no exigió tocar AvionComercial, y la regla de prioridad se implementó modificando solo TorreControlConcreta. En la versión directa ese mismo cambio habría alcanzado a todas las clases de aeronave.
- **Costos y complejidad:** la torre concentra la lógica y es la clase que más crece; si se sumaran reglas como meteorología, varias pistas o franjas horarias, terminaría convertida en un objeto monolítico y convendría dividirla. El flujo aeronave → torre → aeronave es menos evidente al leer el código que una llamada directa. Además, la torre es un punto único de falla y, en un sistema concurrente real, tendría que sincronizar el acceso a sus colas; la simulación es de un solo hilo.
- **Cuándo no usarlo:** cuando los objetos que interactúan son pocos y sus relaciones son simples y estables, porque el mediador solo añade una capa de indirección (Refactoring.Guru, s. f.-b).

Tampoco cuando la comunicación es en un solo sentido y basta con notificar eventos; en ese caso Observer es suficiente.

Uso crítico de inteligencia artificial

- **Prompt principal:**

"Diseña una solución orientada a objetos en Java aplicando el patrón Mediator para coordinar el tráfico y las autorizaciones de aterrizaje entre múltiples aeronaves y una torre de control centralizada, evitando la comunicación directa entre los aviones."

- **Prompt de revisión:**

"Revisa el documento guía de la actividad y lo desarrollado en nuestra resolución (código, UML e informe) y dinos qué se puede mejorar para obtener la calificación más alta."

- **Recomendación aceptada y justificación:**

En la primera iteración, la IA propuso centralizar el estado de la pista y la validación de turnos en TorreControlConcreta. Lo aceptamos porque así las aeronaves no toman decisiones de tráfico y cada clase se limita a su rol en el patrón.

En la revisión, la IA observó que el informe afirmaba que agregar aeronaves no afecta a las demás, pero el código tenía un único tipo de colega, de modo que la afirmación no estaba demostrada. Aceptamos incorporar AvionSanitario con prioridad porque convierte un argumento teórico en una extensión verificable y cubierta por pruebas. También aceptamos separar en el UML las asociaciones que antes aparecían agrupadas en una sola línea y mover el registro de las aeronaves al cliente, para no publicar this desde el constructor.

- **Recomendación modificada o rechazada y justificación:**

En la primera iteración, la IA sugirió que cada aeronave mantuviera una lista de las demás para avisarse entre sí ante emergencias en pista. La rechazamos porque contradice el patrón: los colegas no deben conocerse y una emergencia se reporta a la torre.

En la revisión, la IA planteó como alternativa crear un segundo mediador, TorreControlConPrioridad. Lo descartamos porque habría duplicado toda la gestión de colas para cambiar una sola regla. Preferimos que cada aeronave declare si es prioritaria mediante esPrioritaria() y que la torre use ese dato; así se evita preguntar por el tipo con instanceof y se mantiene un único mediador.

Patrón 02: Command – Control remoto domótico

Escenario / Motivación

Una sala de reuniones cuenta con iluminación y un ventilador que se manejan desde un control remoto programable de cuatro botones. El personal de soporte configura qué hace cada botón y puede reconfigurarlo sin reiniciar el sistema. Además de acciones simples, como encender la luz o ajustar el ventilador, se necesitan escenas: «Modo presentación» apaga la luz y baja el ventilador con un solo botón. Como los usuarios se equivocan, el control debe ofrecer un botón Deshacer que revierta la última acción, incluida una escena completa.

El control decide cuándo se ejecuta una acción y los dispositivos saben cómo realizarla. La dificultad está en que el control no debería saber qué dispositivo hay del otro lado ni cómo se revierte cada acción.

Problema de diseño identificado

La solución directa (Figura 4) es un ControlRemotoDirecto con una referencia a cada dispositivo y un método por botón. Funciona para dos acciones, pero escala mal: cada acción o dispositivo nuevo obliga a modificar el control, los botones quedan fijados en tiempo de compilación y no hay dónde registrar lo que se hizo para poder deshacerlo. Si se intenta resolver con un presionarBoton(accion) y un switch, el condicional crece con cada acción y el control sigue dependiendo de todos los dispositivos.

Lo que hay que organizar es la solicitud misma: convertirla en un objeto que el control pueda guardar, ejecutar, registrar y revertir sin conocer su contenido.

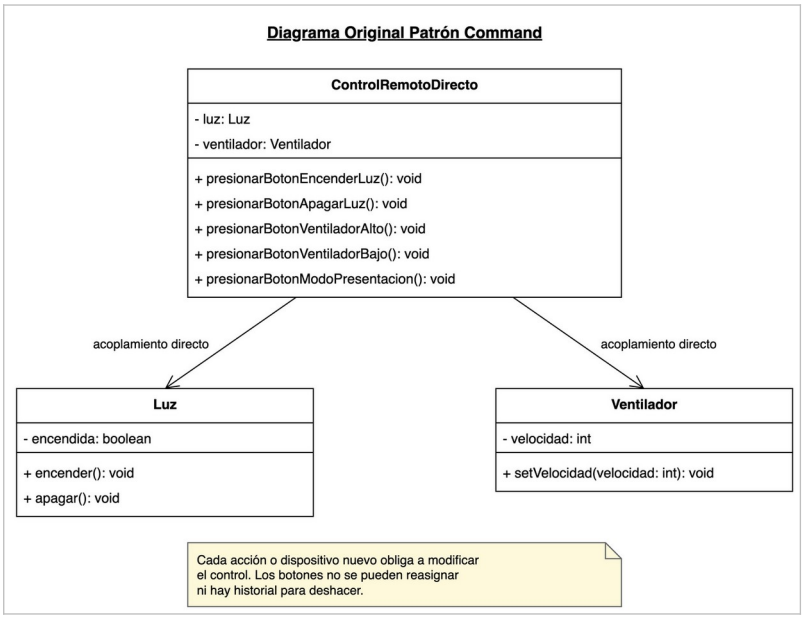


Figura 4: *Diagrama original, sin Command*

Justificación del patrón

Command encapsula una solicitud como objeto, lo que permite parametrizar a quien la invoca con distintas solicitudes, encolarlas o registrarlas, y soportar operaciones que se pueden deshacer (Gamma et al., 1994). Las tres necesidades del escenario corresponden a consecuencias directas del patrón: los botones reasignables son la parametrización del invocador; el deshacer se apoya en el historial de comandos ejecutados, y la escena es un comando compuesto, el MacroCommand que GoF incluye en la descripción del patrón. El caso del control remoto se inspira en el ejemplo clásico de Freeman y Robson (2020), que adaptamos a un escenario de domótica con historial de varios niveles.

Alternativas consideradas y descartadas:

- **Condicionales en el control.** Un presionarBoton(accion) con un switch obliga a editar el control con cada acción nueva y lo mantiene acoplado a todos los dispositivos.
- **Strategy.** La estructura se parece (una interfaz con un método y varias implementaciones), pero la intención es otra: Strategy cambia cómo un objeto realiza su propio trabajo, mientras que aquí cambia qué orden se envía y a qué receptor. Además, Strategy no contempla registrar ni deshacer ejecuciones, que son centrales en este caso.
- **Observer.** El botón avisaría «me presionaron» y el dispositivo decidiría qué hacer. La decisión volvería a quedar en condicionales, ahora dentro del receptor; reasignar un botón implicaría cambiar suscripciones y el deshacer no tendría un lugar natural.
- **Memento.** Podría guardar el estado de cada dispositivo para deshacer, pero obligaría al control a conocer y capturar el estado de todos los dispositivos. Con Command, cada comando guarda solo el dato mínimo que necesita para revertir su propia acción.

Participantes del patrón

- **Command → IComando:** declara ejecutar() y deshacer().
- **ConcreteCommand → EncenderLuzCommand, ApagarLuzCommand y CambiarVelocidadVentiladorCommand:** vinculan una acción con su receptor. Antes de actuar guardan el estado previo del dispositivo para poder revertirlo.
- **ConcreteCommand compuesto → MacroComando:** agrupa varios comandos; los ejecuta en orden y los deshace en orden inverso.
- **Receiver → Luz y Ventilador:** realizan el trabajo y exponen su estado actual.
- **Invoker → ControlRemoto:** guarda un comando por botón (ranura), lo ejecuta al presionarlo y apila cada comando ejecutado en el historial; presionarDeshacer() revierte el último.
- **Client → MainCommandControlLuces:** crea los dispositivos, los comandos y el control, y asigna cada comando a un botón.

La colaboración entre los objetos sigue esta secuencia:

1. El cliente crea los receptores y los comandos concretos, y entrega a cada comando su receptor por el constructor.
2. El cliente asigna los comandos a las ranuras del control con asignarComando(); puede reasignarlas en cualquier momento.
3. Al presionar un botón, ControlRemoto invoca ejecutar() sobre el comando de esa ranura y lo apila en el historial. Si la ranura está vacía, avisa y no registra nada.
4. Al presionar Deshacer, el control desapila el último comando y llama a deshacer(). El comando restaura el estado que guardó; si es una macro, delega en sus comandos en orden inverso.

En la ejecución (Figura 6) se enciende la luz y se sube el ventilador a velocidad 3; luego se activa «Modo presentación» y se deshace como una sola acción, con lo que la sala vuelve exactamente a luz encendida y velocidad 3. Después se reasigna el botón 2 a velocidad baja y, al deshacer hasta vaciar el historial, cada acción se revierte en orden inverso. En ningún momento ControlRemoto conoce a Luz ni a Ventilador.

Diagrama de clases UML

ControlRemoto mantiene dos asociaciones con IComando: ranuras, una agregación (0..*) porque los comandos existen fuera del control y pueden compartirse entre botones, e historial (0..*), que registra lo ejecutado. Los cuatro comandos concretos realizan IComando; cada comando simple navega hacia su receptor con multiplicidad 1 y MacroComando agrega uno o más IComando, lo que expresa la estructura compuesta. Las constantes de Ventilador aparecen subrayadas por ser miembros estáticos. El cliente depende de ControlRemoto e IComando mediante «create», y una nota aclara que también crea los receptores.

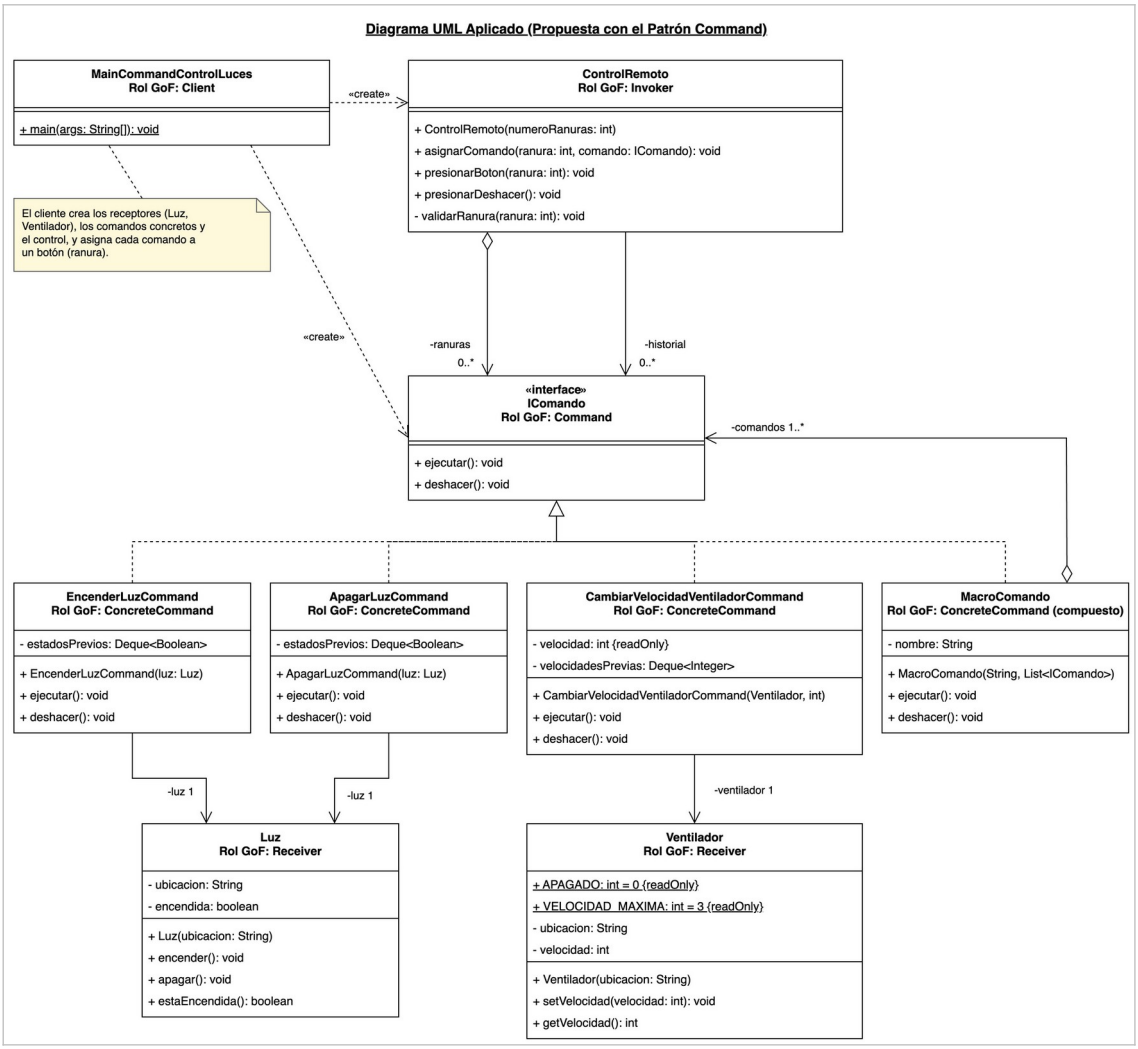


Figura 5: *Diagrama UML con Command*

Código fuente

Repositorio / enlace: https://github.com/carytnas/PDS_Actividad03_Grupo02/tree/main/codigo/ejercicios/src/command

Se muestran el invocador y uno de los comandos concretos:

src/command/ControlRemoto.java

```
package command;


import java.util.ArrayDeque;
import java.util.Deque;


public class ControlRemoto {
  private final IComando[] ranuras;
  private final Deque<IComando> historial = new ArrayDeque<IComando>();


  public ControlRemoto(int numeroRanuras) {
    this.ranuras = new IComando[numeroRanuras];
  }


  public void asignarComando(int ranura, IComando comando) {
    validarRanura(ranura);
    ranuras[ranura] = comando;
```

```
  }

  public void presionarBoton(int ranura) {
    validarRanura(ranura);
    IComando comando = ranuras[ranura];
    if (comando == null) {
      System.out.println("El botón " + ranura + " no tiene ningún comando asignado.");
      return;
    }
    comando.ejecutar();
    historial.push(comando);
  }

  public void presionarDeshacer() {
    if (historial.isEmpty()) {
      System.out.println("No hay acciones para deshacer.");
      return;
    }
    historial.pop().deshacer();
  }

  private void validarRanura(int ranura) {
    if (ranura < 0 || ranura >= ranuras.length) {
      throw new IllegalArgumentException("El control solo tiene los botones 0 a " + (ranuras.length -
1) + ".");
    }
  }
}
```

src/command/EncenderLuzCommand.java

```
package command;


import java.util.ArrayDeque;
import java.util.Deque;


public class EncenderLuzCommand implements IComando {
  private final Luz luz;
  // Pila y no un solo campo: el mismo comando puede ejecutarse varias veces antes de deshacerse.
  private final Deque<Boolean> estadosPrevios = new ArrayDeque<Boolean>();


  public EncenderLuzCommand(Luz luz) {
    this.luz = luz;
  }


  @Override
  public void ejecutar() {
    System.out.println("[Comando] Ejecutando orden: Encender luz.");
    estadosPrevios.push(luz.estaEncendida());
    luz.encender();
  }


  @Override
  public void deshacer() {
    System.out.println("[Comando] Deshaciendo: Encender luz.");
    if (!estadosPrevios.pop()) {
      luz.apagar();
    }
  }
}
```

Un detalle de implementación merece mención. Cada comando guarda sus estados previos en una pila y no en un único campo, porque el mismo objeto puede ejecutarse varias veces antes de deshacerse (por ejemplo, si se presiona dos veces el mismo botón). Con un solo campo, la segunda ejecución sobrescribiría el estado de la primera y el segundo deshacer dejaría la luz en un estado incorrecto. Como el historial del control es LIFO, las pilas de cada comando se vacían en el mismo orden.

```
=== 1. LLEGADA A LA SALA: LUZ Y VENTILADOR ===
[Comando] Ejecutando orden: Encender luz.
-> [Luz sala de reuniones] ENCENDIDA.
[Comando] Ejecutando orden: Ventilador a velocidad 3.
-> [Ventilador sala de reuniones] Velocidad 3.

=== 2. ESCENA "MODO PRESENTACIÓN" ===
[Macro] Activando escena: Modo presentación.
[Comando] Ejecutando orden: Apagar luz.
-> [Luz sala de reuniones] APAGADA.
[Comando] Ejecutando orden: Ventilador a velocidad 1.
-> [Ventilador sala de reuniones] Velocidad 1.

=== 3. DESHACER LA ESCENA ===
[Macro] Deshaciendo escena: Modo presentación.
[Comando] Deshaciendo: Ventilador a velocidad 1.
-> [Ventilador sala de reuniones] Velocidad 3.
[Comando] Deshaciendo: Apagar luz.
-> [Luz sala de reuniones] ENCENDIDA.

=== 4. REASIGNAR EL BOTÓN 2 EN EJECUCIÓN ===
[Comando] Ejecutando orden: Ventilador a velocidad 1.
-> [Ventilador sala de reuniones] Velocidad 1.

=== 5. DESHACER HASTA VACIAR EL HISTORIAL ===
[Comando] Deshaciendo: Ventilador a velocidad 1.
-> [Ventilador sala de reuniones] Velocidad 3.
[Comando] Deshaciendo: Ventilador a velocidad 3.
-> [Ventilador sala de reuniones] APAGADO.
[Comando] Deshaciendo: Encender luz.
-> [Luz sala de reuniones] APAGADA.
No hay acciones para deshacer.
```

Figura 6: *Salida de MainCommandControlLuces*

Validación de la solución

- **Qué resolvió:** el control quedó desacoplado de los dispositivos, un botón puede cambiar de acción en ejecución y cualquier acción, incluida una escena, se puede deshacer.
- **Qué mejoró respecto de la solución directa:** incorporar el ventilador consistió en crear un receptor y un comando nuevos; ControlRemoto no se modificó y no contiene condicionales. Lo mismo ocurrió con la escena: MacroComando combina comandos existentes sin cambiarlos. En la versión directa cada uno de esos cambios habría tocado el control.
- **Costos y complejidad:** una clase por acción, una indirección adicional en cada llamada y una responsabilidad nueva para cada comando: saber revertirse. El historial crece sin límite y en un dispositivo real habría que acotarlo, por ejemplo a las últimas veinte acciones. El deshacer supone, además, que nadie modifica el dispositivo por fuera del control; si alguien enciende la luz desde la pared, el estado guardado deja de ser confiable.
- **Cuándo no usarlo:** cuando hay una sola operación fija que no se reasigna, no se registra ni se deshace; ahí una llamada directa es más clara (Refactoring.Guru, s. f.-a). Tampoco cuando lo único que varía es el algoritmo interno de un objeto, que es el terreno de Strategy.

Uso crítico de inteligencia artificial

- **Prompt principal:**

"Diseña una solución orientada a objetos en Java aplicando el patrón Command para modelar un sistema de control remoto que accione dispositivos inteligentes (como luces), desacoplando los botones de la lógica interna de los aparatos mediante la encapsulación de solicitudes en objetos ejecutables."

- **Prompt de revisión:**

"Revisa el documento guía de la actividad y lo desarrollado en nuestra resolución (código, UML e informe) y dinos qué se puede mejorar para obtener la calificación más alta."

- **Recomendación aceptada y justificación:**

En la primera iteración, la IA propuso una interfaz IComando y que cada comando concreto guardara una referencia a su receptor. Lo aceptamos porque ControlRemoto puede ejecutar cualquier comando sin saber qué dispositivo ni qué acción hay detrás.

En la revisión, la IA observó que el escenario inicial (un botón, una luz, encender y apagar) no aprovechaba lo que distingue a Command de Strategy y que el propio informe mencionaba el deshacer como algo pendiente. Aceptamos agregar deshacer() con historial, un segundo receptor y un macrocomando, porque cada uno ejercita una consecuencia distinta del patrón.

- **Recomendación modificada o rechazada y justificación:**

Modificada: la propuesta de deshacer partía de que cada comando guardara el estado anterior del dispositivo en un campo, que es la forma habitual en la literatura. Al revisarla identificamos el caso del mismo comando ejecutado dos veces, en el que ese campo se sobrescribe, y cambiamos el diseño a una pila por comando. Agregamos además una prueba unitaria específica para ese escenario.

Rechazada: en la primera iteración, la IA sugirió poner la lógica directamente en ejecutar(), sin un receptor. La rechazamos porque el comando solo debe delegar: la lógica del dispositivo pertenece a Luz o a Ventilador. De lo contrario, cada comando acumula lógica de dominio y se duplica entre comandos.

Pruebas automatizadas

Para validar el comportamiento, y no solo la salida por consola, escribimos 13 pruebas con JUnit 5 que se ejecutan con mvn test (Figura 7). En Mediator verifican el orden de llegada, la prioridad del vuelo sanitario, el rechazo de solicitudes duplicadas o de aeronaves no registradas y que ninguna aeronave aterrice sin autorización. En Command verifican la ejecución por ranura, la reasignación en ejecución, el deshacer de cada tipo de comando, el deshacer de una macro como una sola acción, que un botón vacío no altere el historial y el rechazo de botones inexistentes.

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running command.ControlRemotoTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.039 s -- in command.ControlRemotoTest
[INFO] Running mediator.TorreControlConcretaTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.009 s -- in mediator.TorreControlConcretaTest
[INFO] Results:
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

Figura 7: *Resultado de las pruebas unitarias (mvn test)*

Síntesis comparativa: Mediator y Command

Criterio	Mediator	Command
Intención (GoF)	Encapsular cómo interactúa un conjunto de objetos	Encapsular una solicitud como objeto
Pregunta que responde	¿Quién coordina a varios objetos que dependen unos de otros?	¿Cómo represento una orden para guardarla, reasignarla o deshacerla?
Dependencia que elimina	Colega ↔ colega	Invoker ↔ Receiver
Dónde queda la lógica	En el mediador: reglas de interacción	En los comandos: qué hacer y cómo revertirlo
Extensión demostrada	AvionSanitario y regla de prioridad	Ventilador, deshacer y MacroComando
Riesgo principal	Mediador monolítico	Proliferación de clases pequeñas
Señal para no usarlo	Pocos objetos con relaciones estables	Operación única que no se reasigna ni se deshace

Ambos patrones reducen el acoplamiento introduciendo un objeto intermedio, pero en direcciones opuestas. Mediator centraliza: reúne en un solo lugar reglas que antes estaban dispersas. Command distribuye: saca la lógica del invocador y la reparte en objetos pequeños e intercambiables. Por eso, al elegir entre ellos, la pregunta no es cuál desacopla más, sino si el problema consiste en coordinar a varios participantes o en representar una petición.

Balance del uso de inteligencia artificial

Usamos la IA en dos momentos: al inicio, para obtener una primera estructura de cada solución, y al final, como revisor crítico del trabajo completo frente a la rúbrica. En ambos casos aplicamos tres filtros antes de aceptar una sugerencia: que respetara los roles GoF, que pudiera verificarse con una prueba ejecutable

y que mantuviera la correspondencia entre el UML y el código. Las propuestas que no superaban alguno de esos filtros se modificaron o se descartaron, como se detalla en cada patrón.

Conclusiones

La elección de cada patrón dependió más de ubicar bien el problema que de conocer el catálogo. En la torre de control el problema era de coordinación: varias aeronaves compiten por un recurso y, sin un punto central, cada una tendría que conocer a las demás. Mediator lo resolvió, y lo demostró al incorporar los vuelos sanitarios, que exigieron cambiar solo la torre. En el control remoto el problema era cómo representar una orden; Command permitió reasignar botones, deshacer acciones y componer escenas sin tocar el invocador.

Ninguno de los dos es gratuito. La torre acumula reglas y, si el sistema creciera, habría que dividirla; Command multiplica las clases y obliga a cada comando a saber revertirse, asumiendo que el dispositivo no cambia por fuera del control. En ambos casos el costo se justifica por la variabilidad del escenario; en un problema pequeño y estable no se justificaría.

La IA fue útil como punto de partida y como revisor, pero ninguna sugerencia entró al proyecto sin contrastarla con los roles del patrón y con una prueba. El caso más ilustrativo fue el deshacer: la solución habitual funcionaba en el ejemplo simple y fallaba en un caso límite que detectamos al revisarla.

Referencias bibliográficas

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3.ª ed.). Addison-Wesley.

Freeman, E., y Robson, E. (2020). *Head First design patterns* (2.ª ed.). O'Reilly Media.

Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

JUnit Team. (s. f.). *JUnit 5 user guide*. https://junit.org/junit5/docs/current/user-guide/

Oracle. (s. f.). *Java documentation*. https://docs.oracle.com/en/java/

Refactoring.Guru. (s. f.-a). *Command*. https://refactoring.guru/es/design-patterns/command

Refactoring.Guru. (s. f.-b). *Mediator*. https://refactoring.guru/es/design-patterns/mediator

Documentos

- Unidad_03_01_PDS.pdf
- Unidad_03_02_PDS.pdf